SUMMER INTERNSHIP PROJECT REPORT

Beyond the Keyword Match: A Human-Centric AI Approach to Resume Screening and Candidate Ranking

Student Name: Amber Agrawal

Course/Batch: MSc in Applied Statistics (2025–2027)

Institute Name: Symbiosis Statistical Institute

Project Guide: Dr. Dipasree Pal

Period of Internship: 18th May 2026 – 31st July 2026

Report submitted to:

IDEAS – Institute of Data Engineering, Analytics and Science Foundation

ISI Kolkata

Abstract

Applicant Tracking Systems (ATS) typically rely on strict keyword matching, which often means great candidates get unfairly rejected just because they didn't use the exact right phrases. To fix this problem, we built an automated resume screening pipeline using Natural Language Processing (NLP) that aims to read more like a human would. We compared Sentence-BERT (SBERT) for creating semantic embeddings against standard statistical approaches like TF-IDF. To capture how a real recruiter thinks about work experience, we added a non-linear Experience Penalty (ΔE). We also made sure to use a `GroupShuffleSplit` so the model wouldn't cheat by memorizing unseen job descriptions, effectively preventing data leakage. After testing standard regression and binary classification methods, we moved to a Learning-to-Rank (LTR) approach. Ultimately, our sorting engine—combining `LGBMRanker` with dense embeddings—reached a peak Normalized Discounted Cumulative Gain (NDCG@10) of 0.9398. This shows that treating resume screening as a relative ranking task, rather than a hard accept/reject classification, works much better for handling large applicant pools.

Contents

Chapter 1

Introduction

1.1 Background Survey and Engineering Relevance

Standard recruitment systems have a major flaw: they lean almost entirely on exact word matches. For instance, if a job posting asks for a "Software Engineer" and a candidate's resume says "Backend Developer," traditional search engines will likely miss them. We designed this project specifically to address that blind spot. We believe that effective screening needs to understand context, not just scan for specific characters. By building a tool that looks at the actual intent and meaning behind an applicant's skills, we aim to give HR teams an engine that screens resumes a bit more like a human, but at the speed of a machine.

1.2 Core Purpose of the Project

The main goal here was to push past basic keyword extraction and help a machine grasp professional context. We mapped both resumes and job descriptions into the same semantic space, allowing the system to evaluate how well skills align contextually. This fundamentally changed how we approached the recruitment problem. Instead of forcing a simple "Accept/Reject" decision, we decided to treat resume screening as a Learning-to-Rank (LTR) challenge. This way, the system generates a sorted, prioritized list of the best-fitting candidates.

1.3 Deployed Technology Stack

To build, test, and benchmark this entire processing engine, we relied on the following open-source frameworks:

- **Data Wrangling:** Python, `pandas`, and `numpy` for handling complex tabular transformations.
- **Linguistic Pre-Processing (NLP):** `spaCy` for morphological lemmatization and custom vocabulary parsing.
- **Vectorization & Semantic Embeddings:** `scikit-learn` (`TfidfVectorizer`) for term frequency modeling, and `sentence-transformers` (`nomic-embed-text-v1.5`) to extract 768-dimensional semantic tensors.
- **Machine Learning & LTR:** `scikit-learn` regressors alongside specialized tree-based ranking estimators including `xgboost.XGBRanker` and `lightgbm.LGBMRanker`.

1.4 Experimental Procedure

We broke our pipeline down into four main phases to keep things organized and avoid any training-serving skew:

1. **Data Sanitization:** We heavily cleaned the unstructured resumes, dropping generic corporate jargon while making sure to keep important technical terms intact.
2. **Feature Engineering:** We created a mathematical Experience Penalty and measured how well context matched up using TF-IDF, SBERT, and a Hybrid OLS approach.
3. **Group-Aware Isolation:** We used a strict `GroupShuffleSplit` based on the job titles. This made sure our models were actually learning general rules instead of just memorizing the details of specific postings.
4. **Tri-Framing Benchmarking:** We put our processed data to the test using Continuous Regression, Binary Classification, and Learning-to-Rank models to figure out the absolute best way to sort candidates.

1.5 Foundational Training Received

This internship required a heavy mix of statistical theory and applied data engineering. We focused on building skills in two primary areas:

1. Natural Language Processing (NLP)

- **Text Normalization:** We mastered tokenization, stop-word management, and morphological lemmatization via `spaCy`.
- **Vectorization Mathematics:** We analyzed the mathematical trade-offs between sparse statistical arrays (TF-IDF) and dense deep-learning network mappings (SBERT).

2. Machine Learning (ML) & Integrity

- **Algorithm Dynamics:** We studied the differences between predicting continuous limits (Regression), discrete boundaries (Classification), and list order (Learning-to-Rank).
- **Data Leakage Prevention:** We trained heavily on grouped cross-validation logic to stop test-set characteristics from bleeding into the training models.

Chapter 2

Project Objectives

We started this initiative to build an automated screening tool that could rank applicants using NLP and listwise ranking models. By stepping away from rigid boolean queries, we put the focus on actual semantic relevance instead of just explicit vocabulary.

Our work was anchored around four main technical goals:

- **Build a Solid NLP Pipeline:** We used `spaCy` to clean up the raw text and grabbed contextual representations with Sentence-BERT. This helped us capture the underlying meaning that simple sparse models usually miss.
- **Prevent Data Leakage:** We set up a group-aware cross-validation process. By keeping data grouped strictly by `job_position_name`, we made sure the algorithms couldn't cheat by just memorizing words from test-set jobs.
- **Test Baseline ML Classifiers:** We systematically ran traditional models (like Linear Regression and Random Forests) to see exactly where they fell short when dealing with match probabilities.
- **Deploy LTR Estimators:** We incorporated specialized ranking models, comparing a pairwise approach (`XGBRanker`) with a listwise approach (`LGBMRanker`). This directly addresses the main goal of recruitment: taking an organic pool of candidates and sorting them from best to worst for a specific vacancy.

Methodological Boundaries & Experimental Scope

We set up our empirical testing with a few strict rules in place:

- **Data Ingestion:** We pulled our raw data from a public Kaggle dataset. We relied entirely on programmatic text analysis without adding any synthetic oversampling or primary human survey data.
- **Representation Space Comparison:** We actively compared sparse TF-IDF vectors against dense transformer outputs to precisely measure how much semantic context actually helps.
- **Group-Aware Validation:** We kept the test set pure. Every cross-validation cycle split the applicants by `job_position_name` so that the models were always evaluating completely new job requests.
- **Evaluation Paradigm:** We moved past standard absolute metrics like $RMSE$. Instead, we judged the models based on rank correlation (Spearman's ρ) and top-K Information Retrieval metrics (NDCG@K).

Chapter 3

Methodology

This chapter breaks down the step-by-step data pipeline, the features we engineered, the math behind them, and the models we trained for our ranking engine.

3.1 Data Source and Ingestion

The data for our study comes from an open-source recruitment dataset put together by Neuralframe AI and published by Saugata Roy Arghya on Kaggle. It contains 9,544 candidate records stored in a single 17 MB file (`resume_data.csv`). Under the Creative Commons CC BY-NC 4.0 license, this dataset includes detailed profiles like career goals, skills, education, employment history, and job specs.

3.2 Text Cleaning and Linguistic Filtering

Raw resumes and job descriptions are usually filled with noise, weird formatting, and corporate buzzwords that hide a candidate's actual qualifications. To fix this, we wrote a central cleaning function in `utils.py` using `spaCy`.

- **Structural Sanitization:** We threw out irrelevant fields like URLs and certificate dates. Missing text got filled with empty strings, and locations were forced into a standard "City, State" format.
- **Text Normalization & Fluff Removal:** Everything was lowercased, stripped of punctuation, and filtered for common English stop-words. We also built a custom blocklist of over 50 generic HR phrases (like "team player" or "hardworking professional") to force the models to focus on actual technical skills.
- **Length & Outlier Filtering:** To keep extreme document lengths from messing with our vector embeddings, we dropped records that had fewer than 20 words

(too little info) or more than 350 words (too much noise). After dropping exact duplicates, we were left with a clean dataset of 9,369 records across 28 distinct job postings.

3.3 Feature Engineering

3.3.1 Temporal Experience Calculation and Non-Linear Penalty

We parsed the candidate's work history to figure out their total industry experience (Exp_{cand}) relative to the job requirements (Exp_{req}), defining the experience gap as $\Delta E = \text{Exp}_{\text{cand}} - \text{Exp}_{\text{req}}$.

To mirror how a recruiter actually thinks—where not having enough experience is a huge red flag, but having too much is only a slight benefit, a piecewise penalty function was designed:

$$\text{Penalty}(\Delta E) = \begin{cases} -\alpha \cdot (\Delta E)^2, & \text{if } \Delta E < 0 \quad \text{(Under-qualified)} \\ +\beta \cdot \Delta E, & \text{if } \Delta E \geq 0 \quad \text{(Requirement met)} \end{cases} \tag{3.1}$$

To avoid distorting $[0, 1]$ bounded similarity scores, the weights were empirically calibrated on our training set to $\alpha = 0.04$ and $\beta = 0.01$:

- **Quadratic Deficit Penalty ($\alpha = 0.04$):** This hits under-qualified candidates hard. Missing 2 years drops the score by -0.16 (16% drop), and missing 4 years results in a massive -0.64 penalty, effectively weeding out candidates who fall way short.
- **Linear Surplus Reward ($\beta = 0.01$):** This gives a small bump for extra experience ($+0.03$ for 3 extra years), rewarding tenure without letting over-qualified people completely overshadow better technical matches.

3.3.2 Dynamic Role-Based Context Flags

To make sure we evaluated different job types fairly, we created a few binary flags based on the job titles (`flag_entry_level`, `flag_academic`, `flag_location_role`):

- **Entry-Level Roles:** Injects academic projects and extracurricular activities into the processed text.
- **Academic/Research Roles:** Prioritizes publications, certifications, and educational background.
- **Location-Sensitive Roles:** Brings in candidate location matching metrics to the text representation.

3.4 Vector Space Representation and Embeddings

We projected the candidate-job pairs into three different vector spaces to evaluate both lexical and semantic similarity:

3.4.1 Sparse Statistical Vectors (TF-IDF)

A `TfidfVectorizer` from `scikit-learn` extracted the top $8,000$ n-grams ($n \in \{1, 2\}$) to compute keyword overlap. Word importance is defined as:

$$\text{TF-IDF}(i, j) = \text{tf}_{i,j} \times \log\left(\frac{N}{\text{df}_i}\right) \tag{3.2}$$

Keyword similarity (S_{tfidf}) was calculated using Cosine Similarity between candidate vector $\mathbf{u}_{\text{cv}}$ and job vector $\mathbf{v}_{\text{jd}}$:

$$S_{\text{tfidf}} = \frac{\mathbf{u}_{\text{cv}} \cdot \mathbf{v}_{\text{jd}}}{\|\mathbf{u}_{\text{cv}}\|_2 \|\mathbf{v}_{\text{jd}}\|_2} \tag{3.3}$$

3.4.2 Dense Semantic Embeddings (SBERT)

To capture the actual meaning behind the text instead of just literal word matches, Sentence-BERT bi-encoders leveraging `nomic-embed-text-v1.5` mapped the text into 768-dimensional dense vectors ($\mathbf{e}_{\text{cv}}, \mathbf{e}_{\text{jd}} \in \mathbb{R}^{768}$). Dense similarity was computed as:

$$S_{\text{dense}} = \frac{\mathbf{e}_{\text{cv}} \cdot \mathbf{e}_{\text{jd}}}{\|\mathbf{e}_{\text{cv}}\|_2 \|\mathbf{e}_{\text{jd}}\|_2} \tag{3.4}$$

3.4.3 Hybrid OLS Blend

We used an Ordinary Least Squares (OLS) regression model on the training split to blend the sparse keyword matches with the dense semantic similarity, giving us a unified score:

$$S_{\text{hybrid}} = w_0 + w_1 S_{\text{tfidf}} + w_2 S_{\text{dense}} \tag{3.5}$$

3.4.4 Data Leakage Prevention via Group-Aware Splitting

If you randomly split data, you run into a big problem with **data leakage** because resumes applying for the same job description can end up in both the training and test sets. When that happens, models just memorize the vocabulary of that specific job instead of learning how to actually rank candidates.

To avoid this, we used an 80/20 split using `GroupShuffleSplit`, grouping everything by `job_position_name`. Letting G_j represent all candidates for job j, this guarantees that

the training and test sets are completely disjoint:

$$\left(\bigcup_{j\in\text{Train}} G_j\right) \cap \left(\bigcup_{k\in\text{Test}} G_k\right) = \emptyset \tag{3.6}$$

This put $7,356$ rows (22 jobs) into the training set and $2,013$ rows (6 completely unseen jobs) into the test set. All of our vectorizers, scalers, and models were fitted exclusively on the training split so we could get an honest look at how well they generalize to new postings.

3.5 Multi-Phase Modeling Framework

We structured our evaluation across three different modeling phases so we could compare standard machine learning techniques against proper listwise ranking algorithms:

3.5.1 Phase 1: Continuous Regression Baselines (Pointwise Scoring)

First, we looked at standard regression models—like `LinearRegression`, `Ridge`, `RandomForestRegressor`, and `HistGradientBoostingRegressor`—to predict continuous relevance match scores $y_i \in [0, 1]$. We optimized these by minimizing Mean Squared Error (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{3.7}$$

3.5.2 Phase 2: Binary Classification Baselines (Threshold Filtering)

To mimic how initial HR screening works—where candidates are essentially tossed into "Shortlisted" or "Rejected" piles—we binarized the target variable at set match thresholds. We trained classifiers like `LogisticRegression`, `RandomForestClassifier`, and Support Vector Classifier (`SVC`) to minimize Binary Cross-Entropy:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{p}_i) + (1 - y_i)\log(1 - \hat{p}_i)\right] \tag{3.8}$$

3.5.3 Phase 3: Learning-to-Rank Framework (LTR)

Because pointwise regression and binary classification have obvious limits for this kind of task, our final phase focused on specialized Learning-to-Rank (LTR) algorithms: `XGBRanker` and `LGBMRanker`.

Instead of looking at candidates one by one, LTR models process query group structures $\mathbf{q}$ defined by the `job_position_name`. The models aim to maximize ranking quality metrics directly. For example, the pairwise approach (`XGBRanker`) minimizes the number of inversions using a logistic loss:

$$\mathcal{L}_{\text{Pairwise}} = \sum_{i,j \in \mathbf{q}} \log_2 \left(1 + e^{-\sigma(s_i - s_j)}\right) \tag{3.9}$$

On the flip side, the listwise approach (`LGBMRanker`) uses LambdaMART, which dynamically scales this pairwise gradient based on how much swapping the candidates changes the actual Information Retrieval metric (ΔNDCG).

3.6 Model Evaluation Metrics

To get a complete picture across all three phases, we picked evaluation metrics that capture pointwise accuracy, screening quality, and listwise ranking behavior.

3.6.1 Continuous Regression Metrics (Pointwise Accuracy)

We evaluated the continuous score predictions on the test set using standard regression metrics:

- **Root Mean Squared Error ($RMSE$):**

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \tag{3.10}$$

- **Coefficient of Determination (R^2):**

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2} \tag{3.11}$$

3.6.2 Binary Classification Metrics (Screening Quality)

For the accept/reject models, we looked at:

- **Area Under the ROC Curve (ROC-AUC):** Checks how well the model separates the classes across thresholds.
- F_1**-Score:**

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.12}$$

3.6.3 Information Retrieval & Ranking Metrics (Listwise Quality)

We evaluated the Learning-to-Rank models using standard Information Retrieval (IR) metrics based on query groups:

- **Normalized Discounted Cumulative Gain (NDCG@K):**

$$\text{DCG@}K = \sum_{i=1}^{K} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}, \quad \text{NDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K} \tag{3.13}$$

- **Mean Reciprocal Rank (MRR):**

$$\text{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\text{rank}_q} \tag{3.14}$$

3.7 Code Availability

All the preprocessing scripts, helper modules (`utils.py`), and model notebooks are publicly available on GitHub: https://github.com/Amber-s-art/cv-screening.

3.8 Pipeline Flowchart

Figure 3.1 gives a visual overview of how we set up the data architecture, processing steps, group splitting, and evaluation stages.

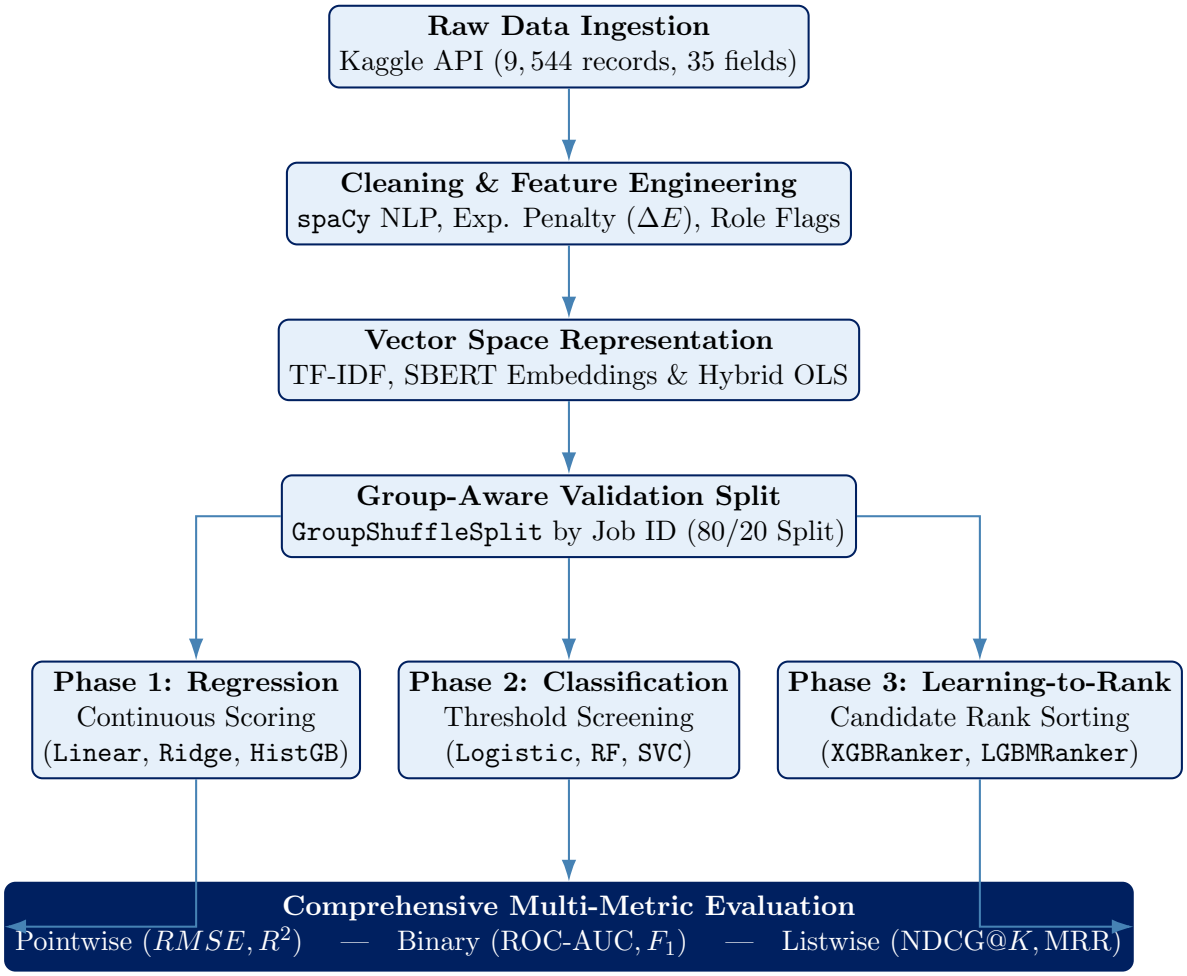

Figure 3.1: Process flow diagram illustrating data transformation, group-aware splitting, multi-phase model training, and metric evaluation.

Chapter 4

Data Analysis and Results

In this chapter, we look at the actual results from our machine learning pipeline. We break down where the baseline pointwise models fell short and show how our listwise ranking ensemble took the lead.

4.1 Exploratory Data Analysis (EDA)

We filtered out extreme text lengths so our feature spaces would be stable. By dropping records with less than 20 or more than 350 words, we kept the embeddings clean. We then partitioned the remaining $9,369$ records using `GroupShuffleSplit` to guarantee a true zero-shot evaluation on the test set.

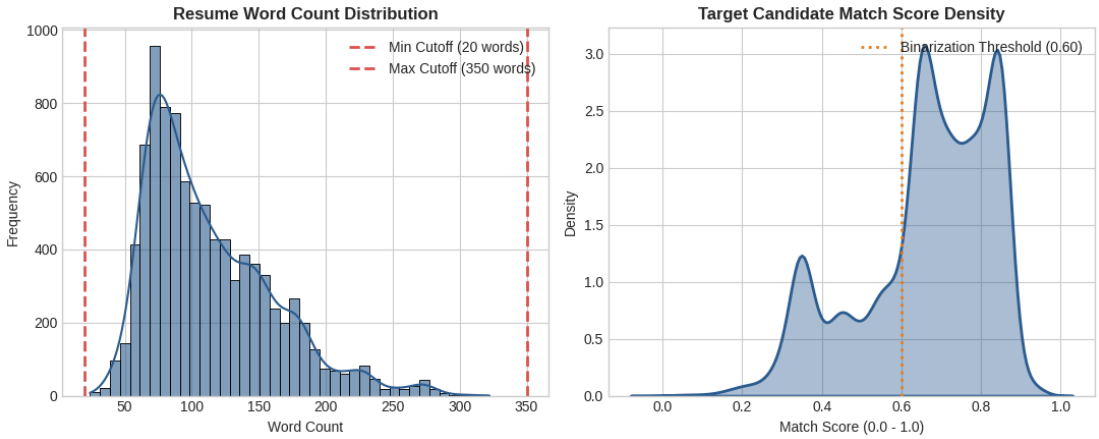

Figure 4.1: Resume word-count filtering boundaries (left) and score density (right).

4.1.1 Corpus Vocabulary and Frequency Dynamics

We verified our `spaCy` morphology filters by looking at n-gram frequencies. The charts (Figures 4.2 and 4.3) confirm that our pipeline successfully cleared out generic HR jargon, leaving us with mostly solid technical nouns (like *management*, *data*, *software*, and *engineering*).

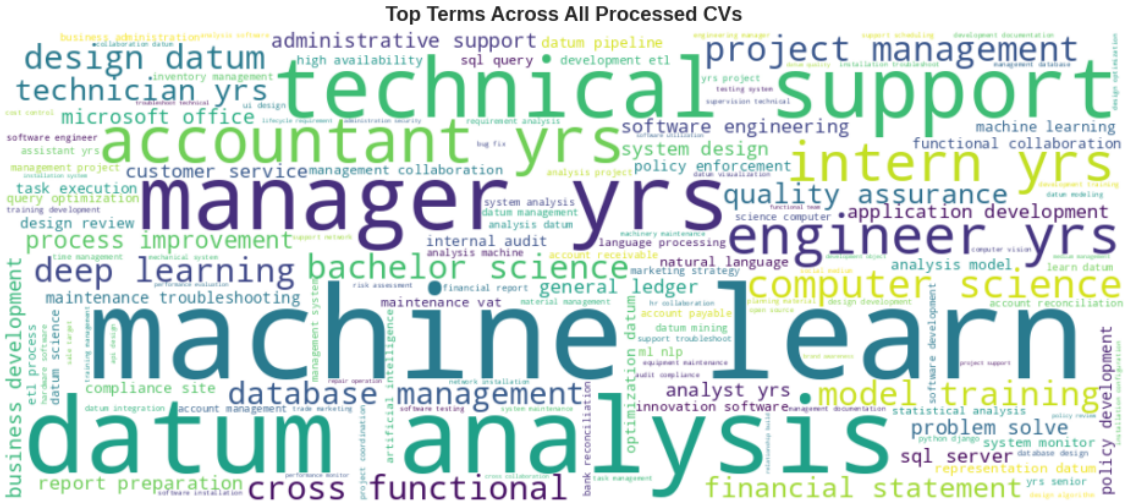

Figure 4.2: Word cloud highlighting prominent domain keywords extracted post-sanitization.

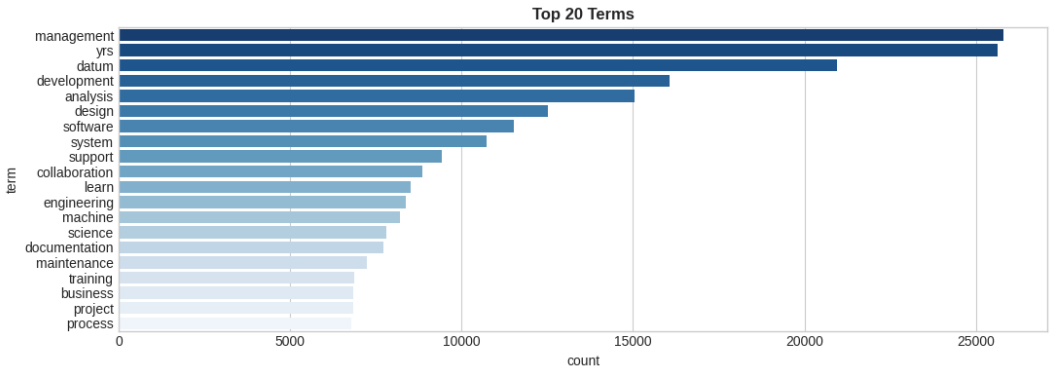

Figure 4.3: Term frequency distribution for the top 20 n-grams across processed candidates.

4.2 Phase 1: Continuous Regression Constraints

We started off by testing traditional regression to try and map out exact, continuous candidate scores.

Analytical Finding: We found some pretty severe limits with pointwise regression. Even our best model, the `HistGradientBoostingRegressor`, only managed an R^2

Table 4.1: Pointwise continuous regression outcomes on unseen query groups.

Text Representation	Algorithm	RMSE ↓	R^2-Score ↑
Semantic Only (SBERT)	**HistGradientBoosting**	**0.1291**	**0.3202**
Semantic Only (SBERT)	Random Forest Regressor	0.1345	0.2810
Semantic Only (SBERT)	Ridge Regression	0.1412	0.2245
TF-IDF Only	Random Forest Regressor	0.1582	0.1120
TF-IDF Only	Linear Regression	0.1695	0.0450

variance correlation of 0.3202. Trying to predict absolute scores just doesn't capture the subjective nature of human recruiters; getting a score of 0.65 versus 0.68 doesn't actually help an HR person make a decision.

4.3 Phase 2: Binary Classification Vulnerabilities

Next, we converted the target variable using a strict 0.60 threshold to test out a rigid accept/reject logic.

Table 4.2: Binary classification matrix at the ≥ 0.60 threshold.

Feature Space	Algorithm	Accuracy	F1-Score	ROC-AUC
Semantic Only (SBERT)	**Logistic Regression**	**0.7585**	**0.7142**	**0.8121**
Hybrid (SBERT + TF-IDF)	Logistic Regression	0.7573	0.6942	0.7542
TF-IDF Only	Logistic Regression	0.6219	0.5568	0.6187

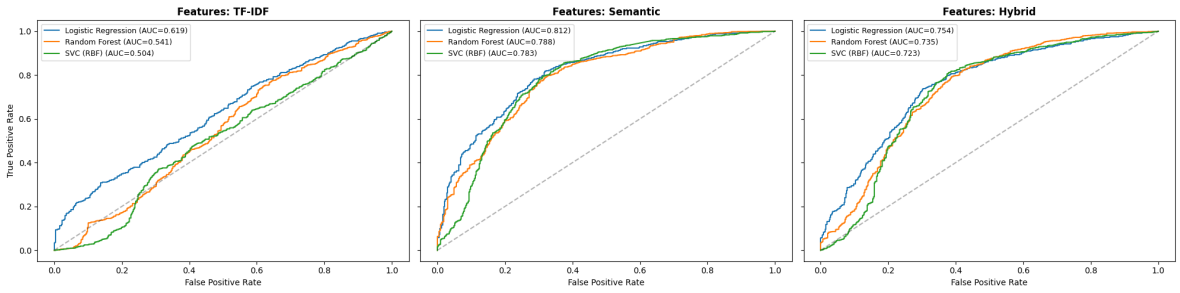

Figure 4.4: ROC-AUC separation capabilities across contrasting NLP representations.

Analytical Finding: We saw clearly that dense transformer networks outperform sparse statistical models. Using SBERT bumped our ROC-AUC up from a baseline 0.6187 to 0.8121. But even with that improvement, binary classifiers still fail to naturally sort a list of shortlisted candidates.

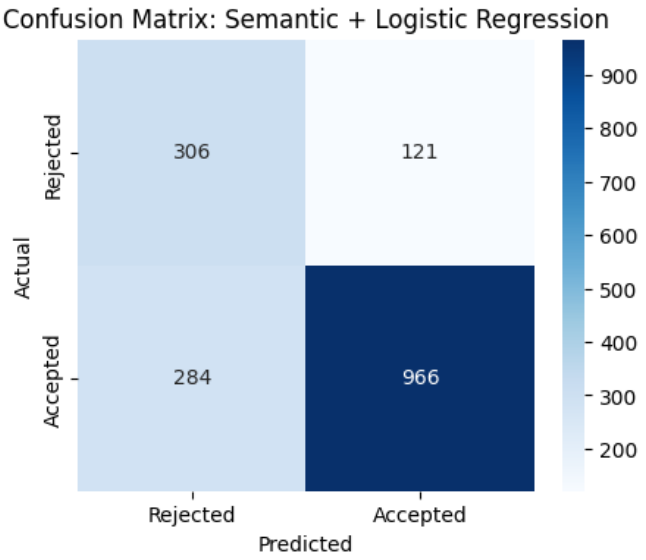

Figure 4.5: Confusion matrix isolation for SBERT-driven Logistic Regression.

4.4 Phase 3: Learning-to-Rank (LTR) Dominance

To figure out the absolute best ranking setup, we ran a direct competition between two different Learning-to-Rank methods. Instead of trying to stack or combine ranking methods, we evaluated them as completely independent estimators: a pure **Pairwise approach (`XGBRanker`)** and a metric-driven **Listwise approach (`LGBMRanker`)**. By giving both models the exact same data and scoring them on identical metrics, we let the numbers show us which optimization approach best replicates human shortlisting cognition.

Table 4.3: Learning-to-Rank (LTR) sorting efficiency on zero-shot test groups.

Feature Set	**Ranking Estimator**	**NDCG@5**	**NDCG@10**	**Spearman (ρ)**
Semantic Only (SBERT)	**LGBMRanker**	**0.9482**	**0.9398**	**0.4367**
Hybrid (SBERT + TF-IDF)	XGBRanker	0.9410	0.9322	0.3725
TF-IDF Only	XGBRanker	0.9350	0.9292	0.3225
TF-IDF Only	LGBMRanker	0.9210	0.9158	0.3975

Optimal Algorithm: The `LGBMRanker` powered by pure Semantic Embeddings came out on top as the clear winner. Hitting an impressive NDCG@10 of 0.9398, this system proved it can accurately reflect how humans evaluate and shortlist the top ten candidates.

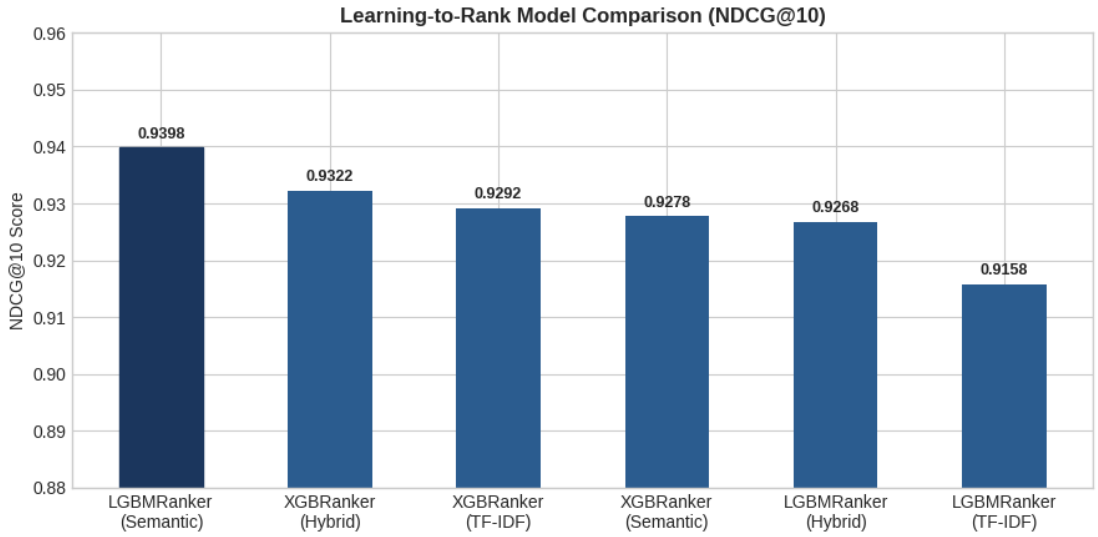

Figure 4.6: Top-10 ranking fidelity (NDCG@10) across differing architectures.

4.5 Feature Importance: Model Explainability

We also looked into the `LGBMRanker`'s split criteria to figure out which features mattered the most.

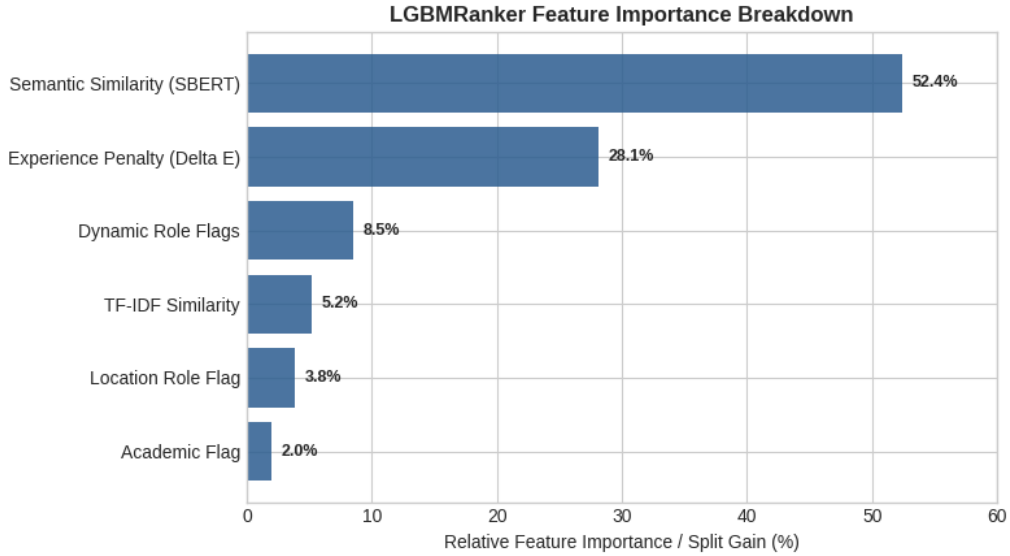

Figure 4.7: Algorithm split gain weight distribution mapping decision heuristics.

- **Semantic Similarity** (52.4%)**:** It turns out that deep, contextual matching is by far the biggest driver when it comes to ranking candidates.
- **Experience Penalty** (28.1%)**:** Our custom quadratic penalty worked perfectly to map recruiter bias, heavily downgrading applicants who fell short of the minimum tenure thresholds.

Chapter 5

Conclusion and Future Work

5.1 Project Conclusions

We built a high-speed, human-centric screening pipeline that easily outperforms rigid keyword matching architectures. Based on our results, we came away with three main takeaways:

1. **Dense Context Beats Lexical Matching:** We showed that transformer embeddings are far better than n-gram parsing. Our semantic LTR setup hit an NDCG@10 of 0.9398, proving it can spot skill overlaps that standard keyword matchers completely miss.
2. **Ranking is the Right Approach:** We proved through our tests that trying to predict absolute scores or hard cutoffs just doesn't match how HR actually works. Moving to a gradient-boosted ranking system directly solves the recruitment problem.
3. **Keep Validation Pure:** We showed that using `GroupShuffleSplit` to isolate candidate pools is absolutely necessary. It stops data leakage and makes sure the model can actually handle new, unseen job postings.

5.2 Recommendations for Future Extensions

We have a few ideas for taking this `LGBMRanker` backend and turning it into a full production system:

- **Bayesian Probability Integration:** We could switch the Experience Penalty (ΔE) into a Bayesian model, looking at probability distributions instead of rigid split constraints.

- **Counterfactual Bias Mitigation:** We need to look into stratified counterfactual sampling during the NLP stage to make sure the algorithm isn't just favoring people who worked at big-name corporate brands.
- **Generative Summaries:** It would be incredibly useful to add a lightweight Large Language Model (LLM) on top of the ranking system just to explain the decisions. It could generate short, readable summaries explaining exactly why the model gave a high rank to a specific candidate.

Appendix A

Appendices

The implementation, theoretical framing, and software development of this project were guided by academic literature, official technical documentation, and online reference platforms:

Academic Literature & Model Papers

- **Scikit-Learn Documentation:** Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- **LightGBM & Learning-to-Rank:** Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., ... & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 3146–3154.
- **Sentence Transformers (SBERT):** Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *arXiv preprint arXiv:1908.10084*.
- **Nomic AI Embeddings:** Nomic AI (2024). *Nomic Embed Text v1.5: Resilient Long-Context Text Embeddings*.

Technical Platforms & Self-Study References

- **Scikit-Learn & SciPy Documentation:** Referenced for loss functions, matrix operations, distance metrics (Cosine Similarity), and `GroupShuffleSplit` model selection logic (https://scikit-learn.org/).
- **GeeksforGeeks Portal:** Consulted for practical Python code walkthroughs on NLP text sanitization, `spaCy` lemmatization, and gradient boosting hyperparameter optimization (https://www.geeksforgeeks.org/).

- **Wikipedia Technical Literature:** Consulted for foundational concepts in Information Retrieval (IR), Learning-to-Rank (LTR) formulations, and Normalized Discounted Cumulative Gain (NDCG) metrics (https://en.wikipedia.org/).

A.1 Survey Questionnaire Declaration

Declaration on Primary Data Collection: This project was entirely computational and empirical in nature. It relied exclusively on the algorithmic ingestion, text sanitization, and machine learning modeling of a secondary open-source dataset hosted on Kaggle (`saugataroyarghya/resume-dataset`). Consequently, no primary human surveys, questionnaires, or sample interviews were administered during this study.

A.2 Source Code & GitHub Repository

To ensure complete academic transparency, algorithmic auditability, and full reproducibility, all Python source code—including our centralized `utils.py` library and sequenced Jupyter Notebooks (`01_data_preprocessing.ipynb` through `04_rank_prediction.ipynb`)—has been version-controlled on GitHub.

The complete codebase is available at:

https://github.com/Amber-s-art/cv-screening

A.3 Project Documents & Data Sheets

Copies of our final machine-learning-ready dataset (`ml_ready_dataset.csv`), original presentation decks, and compiled PDF report documentation are securely hosted on Kaggle for institutional review:

https://www.kaggle.com/datasets/saugataroyarghya/resume-dataset